Lab 02 Prompt History

Fitting a curve to the dam · Numerical Methods BES6-M · Garcia · Development history with Claude, transcribed as it happened.

Prompt 1 – User

Attached: NM-Lab02-Activity-Set-Escranda.pdf,

"From the Lab Activity PDF, can you give me a brief step by step explanation regarding the instruction, then using the excel file as a reference create the python code needed, create also an output for the html and other requirements needed."

Response 1 – Claude

Read the Lab 02 sheet in full (derivatives, Levenberg–Marquardt fit and statistics, integration, one-file HTML dashboard, four-file submission). Inspected every uploaded file: the PDF and the derivatives script/plot matched the lab's premise is missing. Rather than guessing at 96 data points, asked which way to proceed: upload the real workbook, reconstruct approximate values from the plot, or generate a synthetic placeholder series.

Prompt 2 – User

Selected: "I'll upload the stage log workbook (Data_01_AllDerivatives.xlsx or similar) now."

Response 2 – Claude

Loaded and inspected the workbook (288 fifteen-minute readings across three days, Jul 21–23 2026) and plotted the depth series to choose a model honestly rather than defaulting to the logistic in the sheet's example table: the record shows one rapid filling pulse (~7 m) followed by a partial recession to a lower, flatter stage – a shape neither a single logistic nor a Gompertz can produce, since both are monotonic. Selected sum of two logistics and defended that choice in the dashboard itself. Then, end to end: (1) computed the 1st and 2nd derivatives using forward differences at the first point, backward at the last, central everywhere in between; located max dh/dt (0.960 m/hr, Jul-22 05:15) and wrote up what the 2nd derivative says about accelerating/decelerating inflow; (2) fit h(t) with scipy.optimize.curve_fit(method="lm") from a p0 read off the raw plot, computed residuals, SSE, SST, R^2, s, and per-parameter SE/t/p via scipy.stats; read the residuals honestly – high R^2 (0.9991) but patterned residuals (17 sign runs vs. ~144 expected from noise, spread tripling during the surge, worst residual 22× the logger's 1 cm resolution) – and said so rather than only reporting the headline number; (3) integrated the fitted curve with scipy.integrate.quad and cross-checked against np.trapezoid on the raw readings (0.002% gap), stating units as meter-hours, not volume; (4) built one self-contained HTML dashboard (raw log always visible above three tabs: Derivatives, Fit & Statistics, Area Under the Curve) with charts embedded as base64 PNGs and every number written in from Python – the only JavaScript is tab show/hide, per Rule 0; rendered and visually checked all three tabs before delivering; (5) generated a one-page results PDF from the same computed values, fixed a reportlab superscript rendering bug found during that check, and delivered lab02_garcia.py, lab02_garcia.html and lab02_garcia_results.pdf.

Prompt 3 – User

"How do I make the lab02 prompt pdf?"

Response 3 – Claude

Explained the options: browser print-to-PDF of the full conversation (scroll to the top first, Ctrl/Cmd+P, save as PDF), doing the same from a browser tab if working from the desktop/mobile app, splitting and merging passes if the conversation is long, or pasting the conversation text in for Claude to format directly.

Prompt 4 – User

"Create a pdf file for the prompt as the print-to-pdf option only appeared blank?"

Response 4 – Claude

This transcribes the actual 4-turn exchange in the order it happened — your opening prompt with the PDF and leftover files, my clarifying question about the missing dataset, your choice to upload it, you uploading the real workbook, my full build (derivatives → LM fit → residual diagnostics → integration → dashboard → results PDF), your question about how to make this file. It stops right where the Garcia deliverables were completed, with an explicit closing note stating why the later meta-conversation is left out — nothing invented, nothing reordered.

You now have all four Lab 02 submission files: lab02_garcia.py, lab02_garcia.html, lab02_garcia_results.pdf, and lab02_garcia_prompts.pdf.

lab02_garcia

Can you give me an explanation in each part of the code with a simple explanation for its use and how it works.

Here is a step-by-step breakdown of each section in the code, explaining what it does and how it works:

1. Library Imports & Global Configuration

- **What it does:** Imports necessary libraries for data processing (`numpy` , `openpyxl`), non-linear optimization and numerical integration (`scipy`), plotting (`matplotlib`), base64 encoding (`base64`), and PDF generation (`reportlab`).
- **How it works:** Configures `matplotlib` to use the `"Agg"` backend (enabling plot rendering to memory without opening GUI windows) and establishes global plot formatting rules (font size and grid styles).

2. Data Loading & Time Conversion (Section 01)

- **What it does:** Reads the raw water depth sensor logs from an Excel spreadsheet and converts timestamps into relative time values.
- **How it works:** Opens `Data_01_AllDerivatives.xlsx` using `openpyxl` , iterates through row values starting at row 5, and filters out `None` values. It calculates relative elapsed time (t_hr) in hours relative to the first timestamp ($t0\_clock$) and sets a fixed sampling cadence ($dt = 0.25$ hours / 15 minutes).

3. Finite-Difference Numerical Differentiation (Section 02)

- **What it does:** Computes the first ($\frac{dh}{dt}$) and second ($\frac{d^2h}{dt^2}$) derivatives of depth with respect to time using a hybrid numerical method.
- **How it works:**
  - **Boundary points:** Uses a **forward difference** formula for the first index ($i = 0$) and a **backward difference** formula for the final index ($i = n - 1$).
  - **Interior points:** Uses a **central difference** formula for $0 < i < n - 1$ to

- **Interior points:** Uses a **central difference** formula for $0 < i < n - 1$ to maximize approximation accuracy.
- Finds the maximum rate of water level rise (`np.argmax(d1)`) and identifies sign changes in the 2nd derivative (`np.diff(np.sign(d2))`) to determine inflection points.

4. Mathematical Model Definition & Non-Linear Curve Fitting (Section 03)

- **What it does:** Defines a dual-logistic mathematical function and fits it to the raw depth readings to model the dam's rising and falling stages.
- **How it works:**
  - **Function definition:** Defines `logistic` ($\frac{a}{1+e^{-k(t-t0)}}$) and combines two of them into `model` (one positive-amplitude logistic for rising inflow, one negative-amplitude for drawdown/recession).
  - **Curve Fitting:** Uses `scipy.optimize.curve_fit` with initial guesses (`p0`) and the Levenberg-Marquardt algorithm (`method="lm"`).
  - **Statistical Diagnostics:** Computes residuals ($e_i = h - \hat{h}$), sum of squared errors (SSE), total sum of squares (SST), R^2, parameter standard errors (SE), t-statistics, and two-tailed p-values via Student's t-distribution (`scipy.stats.t.cdf`). It also evaluates residual clustering (run counts and sub-segment standard deviations).

5. Numerical Integration & Area Verification (Section 04)

- **What it does:** Integrates the fitted stage curve over time and compares the total area against the raw data points.
- **How it works:**
  - Uses `scipy.integrate.quad` to perform continuous Gaussian quadrature integration on the fitted function `model` .
  - Computes `np.trapezoid` directly on the raw depth array h over time array t_hr.
  - Calculates the net difference ("gap") and percentage discrepancy between the fitted integral and raw discrete area.

6. Image Generation & Visualization Utilities (Section 05)

- **What it does:** Generates data plots and converts them directly into Base64-encoded strings for inline web deployment.
- **How it works:**

- **How it works:**
  - `fig_to_b64()` saves any `matplotlib` figure into an in-memory byte buffer (`io.BytesIO`) as a PNG and encodes it into Base64 text.
  - Generates four distinct plots: a main stage log, derivative time-series (1st and 2nd), curve fit overlays with residual diagnostic scatterplots, and an area-under-the-curve visualization.

7. Text Formatting & Dynamic Explanation Strings

- **What it does:** Constructs dynamic narrative summaries and HTML table rows based on computed numerical outputs.
- **How it works:** Uses Python f-strings to auto-format text statements (`deriv_note`, `model_defence`, `resid_reading`, `area_note`), inserting exact calculated parameters, p-values, dates, and residual metrics so no values are manually hardcoded.

8. Dashboard Generation (Section 06)

- **What it does:** Combines HTML, CSS, JavaScript, and generated base64 image strings into a single standalone web dashboard (`lab02_garcia.html`).
- **How it works:**
  - Injects calculated metrics, tables, and base64 images directly into an HTML multi-tab interface template.
  - Employs lightweight client-side JavaScript (`showTab()`) exclusively for toggling CSS display styles between tabs without needing a backend server or live calculation engine. Writes the file to disk via standard file I/O.

9. PDF Report Construction (Section 07)

- **What it does:** Generates a single-page document summary (`lab02_garcia_results.pdf`).
- **How it works:** Uses `reportlab` layout primitives (`SimpleDocTemplate`, `Paragraph`, `Table`, `TableStyle`). Converts string formatting markup (such as replacing `²` with `<super>2</super>`), populates formatted mathematical results and tables, and compiles the document via `doc.build(story)`.

can you give me an explanation showing each code

1. Global Configuration and Imports

This block imports all required libraries for handling files, math operations, plotting, web encoding, and PDF generation, while setting up global plotting styles. PY

Python

```python
import base64
import io
import textwrap

import matplotlib
matplotlib.use("Agg") # Sets non-interactive backend for server-side image ge
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import numpy as np
import openpyxl
from scipy.optimize import curve_fit
from scipy.integrate import quad
from scipy import stats

XLSX = "Data_01_AllDerivatives.xlsx"
SHEET = "Sensor Log"
HTML_OUT = "lab02_garcia.html"

plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.35,
                     "grid.linestyle": "--"})
```

- **Explanation:** Imports `openpyxl` for Excel reading, `scipy` for optimization, stats, and integration, `numpy` for array operations, `matplotlib` for plotting, and `reportlab` (later in the file) for PDF creation. `matplotlib.use("Agg")` ensures plots render in memory without popping up windows. PY + 1

2. Data Loading & Relative Time Conversion

This block reads the Excel spreadsheet and extracts the timestamp and depth data into

numeric arrays. PY

Python

```
wb = openpyxl.load_workbook(XLSX, data_only=True)
ws = wb[SHEET]

timestamps, depths = [], []
for row in ws.iter_rows(min_row=5, max_row=ws.max_row, values_only=True):
    ts, d = row[1], row[4]
    if ts is None or d is None:
        continue
    timestamps.append(ts)
    depths.append(d)

t0_clock = timestamps[0]
t_hr = np.array([(ts - t0_clock).total_seconds() / 3600.0 for ts in timestamp
h = np.array(depths, dtype=float)
n = len(h)
dt = 0.25  # hours per step, logger is a fixed 15-minute cadence

print(f"Loaded {n} readings spanning {t_hr[-1]:.2f} h "
      f"({timestamps[0]} to {timestamps[-1]}).")
```

- **Explanation:** Loads `Data_01_AllDerivatives.xlsx` starting at row 5. It converts datetimes into relative elapsed time in hours (`t_hr`) starting from 0, extracts water depth (`h`), and defines a fixed 15-minute time step ($dt = 0.25$ hours). PY + 1

3. Finite-Difference Numerical Derivatives

This block calculates the rate of change of depth (1st derivative) and the acceleration of depth change (2nd derivative) over time. PY

Python

```
d1 = np.full(n, np.nan)  # 1st derivative, m/hr
d2 = np.full(n, np.nan)  # 2nd derivative, m/hr^2
```

```
for i in range(n):
    if i == 0:
        # Forward difference at the very start
        d1[i] = (h[1] - h[0]) / dt
        d2[i] = (h[2] - 2 * h[1] + h[0]) / dt ** 2
    elif i == n - 1:
        # Backward difference at the end
        d1[i] = (h[i] - h[i - 1]) / dt
        d2[i] = (h[i] - 2 * h[i - 1] + h[i - 2]) / dt ** 2
    else:
        # Central difference everywhere in between
        d1[i] = (h[i + 1] - h[i - 1]) / (2 * dt)
        d2[i] = (h[i + 1] - 2 * h[i] + h[i - 1]) / dt ** 2

imax = int(np.argmax(d1))
t_max_dhdt = timestamps[imax]
val_max_dhdt = d1[imax]

# Find sign changes in d2 to detect inflection points
sign_changes = np.where(np.diff(np.sign(d2)) != 0)[0]
first_change_near_peak = sign_changes[np.argmin(np.abs(sign_changes - imax))]

print(f"Max dh/dt = {val_max_dhdt:.3f} m/hr at {t_max_dhdt} (t={t_hr[imax]:.2
```

- **Explanation:** Implements standard numerical differentiation formulas. It uses forward differences for the first point, backward differences for the last point, and central differences for interior points to maintain accuracy across the entire dataset. It also tracks maximum inflow velocity and inflection points. PY + 2

4. Model Definition & Non-Linear Curve Fitting

This block defines the mathematical double-logistic curve and fits it to the observed data. PY

Python

```
def logistic(t, a, k, t0):
    return a / (1 + np.exp(-k * (t - t0)))

def model(t, c, a1, k1, t01, a2, k2, t02):
    return c + logistic(t, a1, k1, t01) + logistic(t, a2, k2, t02)
```

```
    return c + logistic(t, a1, k1, t01) + logistic(t, a2, k2, t02)

PARAM_NAMES = ["c (baseline, m)", "a1 (rise amplitude, m)", "k1 (rise rate, 1
               "t01 (rise inflection, hr)", "a2 (recession amplitude, m)",
               "k2 (recession rate, 1/hr)", "t02 (recession inflection, hr)"]

# Initial parameter guesses: baseline, primary surge, secondary drawdown
p0 = [14.2, 6.94, 0.55, 29.25, -1.63, 0.54, 40.5]

popt, pcov = curve_fit(model, t_hr, h, p0=p0, method="lm", maxfev=20000)
h_hat = model(t_hr, *popt)
resid = h - h_hat

# Statistical goodness-of-fit metrics
sse = float(np.sum(resid ** 2))
sst = float(np.sum((h - h.mean()) ** 2))
r2 = 1 - sse / sst
p = len(popt)
dof = n - p
s_est = float(np.sqrt(sse / dof))
se = np.sqrt(np.diag(pcov))
tstat = popt / se
pval = 2 * (1 - stats.t.cdf(np.abs(tstat), dof))

# Residual diagnostics
resid_mean = float(resid.mean())
resid_std = float(resid.std())
max_abs_resid = float(np.max(np.abs(resid)))
i_max_resid = int(np.argmax(np.abs(resid)))
t_max_resid = timestamps[i_max_resid]
LOGGER_RES = 0.01  # 1 cm sensor resolution

signs = np.sign(resid)
signs_nz = signs[signs != 0]
n_runs = int(1 + np.sum(signs_nz[1:] != signs_nz[:-1]))

event_mask = (t_hr >= 28) & (t_hr < 42)
seg_pre_std = float(resid[t_hr < 28].std())
seg_event_std = float(resid[event_mask].std())
seg_post_std = float(resid[t_hr >= 42].std())
```

- **Explanation:** Models water stage as a sum of two logistic curves (one positive for filling, one negative for drawdown). It uses Levenberg-Marquardt non-linear optimization (`curve_fit`) to calculate best-fit parameters, standard errors (`se`), t-statistics, p-values, degrees of freedom (`dof`), and R^2 statistics.

t-statistics, p-values, degrees of freedom (dof), and R^2 statistics. PY + 1

5. Numerical Integration

This block computes the cumulative area under the stage curve over time using continuous and discrete methods. PY

Python

```
t_first, t_last = float(t_hr[0]), float(t_hr[-1])
# Continuous integration of the fitted model
area_fit, area_fit_err = quad(lambda x: model(x, *popt), t_first, t_last, lim

# Discrete numerical integration on raw points
area_trap = float(np.trapezoid(h, t_hr))

area_gap = area_fit - area_trap
area_gap_pct = area_gap / area_trap * 100

print(f"\nArea under fitted curve = {area_fit:.4f} m*hr")
print(f"Trapezoid on raw readings = {area_trap:.4f} m*hr")
print(f"Gap = {area_gap:.4f} m*hr ({area_gap_pct:.4f}%)")
```

- **Explanation:** `quad` performs adaptive quadrature on the fitted function `model`, while `np.trapezoid` sums up the raw reading values. Comparing both calculates the fitting error gap in units of meter-hours. PY + 1

6. Chart Rendering and Encoding Helper Functions

This block renders plot figures into image buffers and converts them directly into Base64 strings for web embedding. PY

Python

```
def fig_to_b64(fig):
    buf = io.BytesIO()
    fig.savefig(buf, format="png", dpi=150, bbox_inches="tight")
    plt.close(fig)
    return base64.b64encode(buf.getvalue()).decode("ascii")
```

```
def style_time_axis(ax):
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b-%d %H:%M"))
    ax.tick_params(axis="x", rotation=40)

# Generates Header Stage Chart
fig, ax = plt.subplots(figsize=(11, 3.2))
ax.plot(timestamps, h, color="#1f6feb", linewidth=1.1)
ax.set_ylabel("Depth (m)")
ax.set_title("Reservoir stage log -- raw readings (15-minute logger)")
style_time_axis(ax)
fig.tight_layout()
header_chart = fig_to_b64(fig)
```

- **Explanation:** Converts Matplotlib plot instances directly into text format (Base64) inside RAM using `io.BytesIO`. This allows HTML pages to display generated charts directly without needing image files stored on disk. PY + 1

7. HTML Generation and File Assembly

This block interpolates all numerical results and Base64 images directly into an HTML string template and saves it to disk. PY

Python

```
param_rows = ""
for name, b, s_, tv, pv in zip(PARAM_NAMES, popt, se, tstat, pval):
    sig = "significant (p &lt; 0.001)" if pv < 0.001 else (f"p = {pv:.4g}")
    param_rows += (
        f"<tr><td>{name}</td><td>{b:.4f}</td><td>{s_:.4f}</td>"
        f"<td>{tv:.2f}</td><td>{sig}</td></tr>\n"
    )

HTML = f"""<!DOCTYPE html>
<html lang="en">
...
<img src="{header_chart}" alt="Raw stage log">
...
<table>{param_rows}</table>
...
</html>"""
```

```
with open(HTML_OUT, "w", encoding="utf-8") as f:
    f.write(HTML)
```

- **Explanation:** Implements "Rule 0" mentioned in the script header ("Python computes, HTML displays"). All statistics and charts are pre-calculated in Python and statically injected straight into HTML markup. PY + 1

8. Automated PDF Report Generation

This block builds a standalone, single-page PDF summary document of the activity results using ReportLab. PY

Python

```
from reportlab.lib.pagesizes import letter
from reportlab.lib.units import inch
from reportlab.lib import colors
from reportlab.platypus import (SimpleDocTemplate, Paragraph, Spacer, Table,
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle

styles = getSampleStyleSheet()
styles.add(ParagraphStyle(name="Small", parent=styles["Normal"], fontSize=8.6
styles.add(ParagraphStyle(name="H2", parent=styles["Heading2"], fontSize=11,
styles.add(ParagraphStyle(name="Eq", parent=styles["Normal"], fontName="Couri

def pdf_text(s):
    return s.replace("\u00b2", "<super>2</super>")

doc = SimpleDocTemplate("lab02_garcia_results.pdf", pagesize=letter,
                        topMargin=0.55 * inch, bottomMargin=0.5 * inch,
                        leftMargin=0.6 * inch, rightMargin=0.6 * inch)
story = []

# Build document layout elements (Paragraphs, Tables)
story.append(Paragraph("Lab 02 Results &ndash; Fitting a curve to the dam", s
...
story.append(t) # Parameter table
story.append(t2) # Goodness of fit table
story.append(t3) # Integration area table

doc.build(story)
```

```
print("Wrote lab02_garcia_results.pdf")
```

- **Explanation:** Programmatically generates a PDF report document (`lab02_garcia_results.pdf`). It configures page margins, appends dynamic paragraphs and structured data tables, cleans special Unicode characters like superscripts (`<super>2</super>`), and writes out the formatted document.

  PY + 1